COMPUTER VISION

Ms. Jayshri Todase

Course Objectives

- To understand the **fundamental concepts of computer vision** and its **applications**.
- To study **image processing techniques** essential for computer vision tasks.
- To explore **feature extraction, object detection, and recognition methodologies.**
- To implement and evaluate **machine learning** and **deep learning** approaches **in computer vision.**
- To analyze real-world challenges and advancements in computer vision technologies.

Course Overview

Course Code: **BTDS-213**

Semester: 4th sem (2nd Year) BTech

Credits: **2**

In Sem Evaluation = **25 Marks**

Mid Sem Exam = **25 Marks**

End Sem Exam = **50 Marks**

Syllabus

Unit I -Introduction to Computer Vision

Unit II - Image Processing Fundamentals

Unit III - Feature Detection and Description

Unit IV - Object Detection and Recognition

Unit V - Machine Learning and Deep Learning for Computer Vision

Unit VI - Challenges and Future Trends in Computer Vision

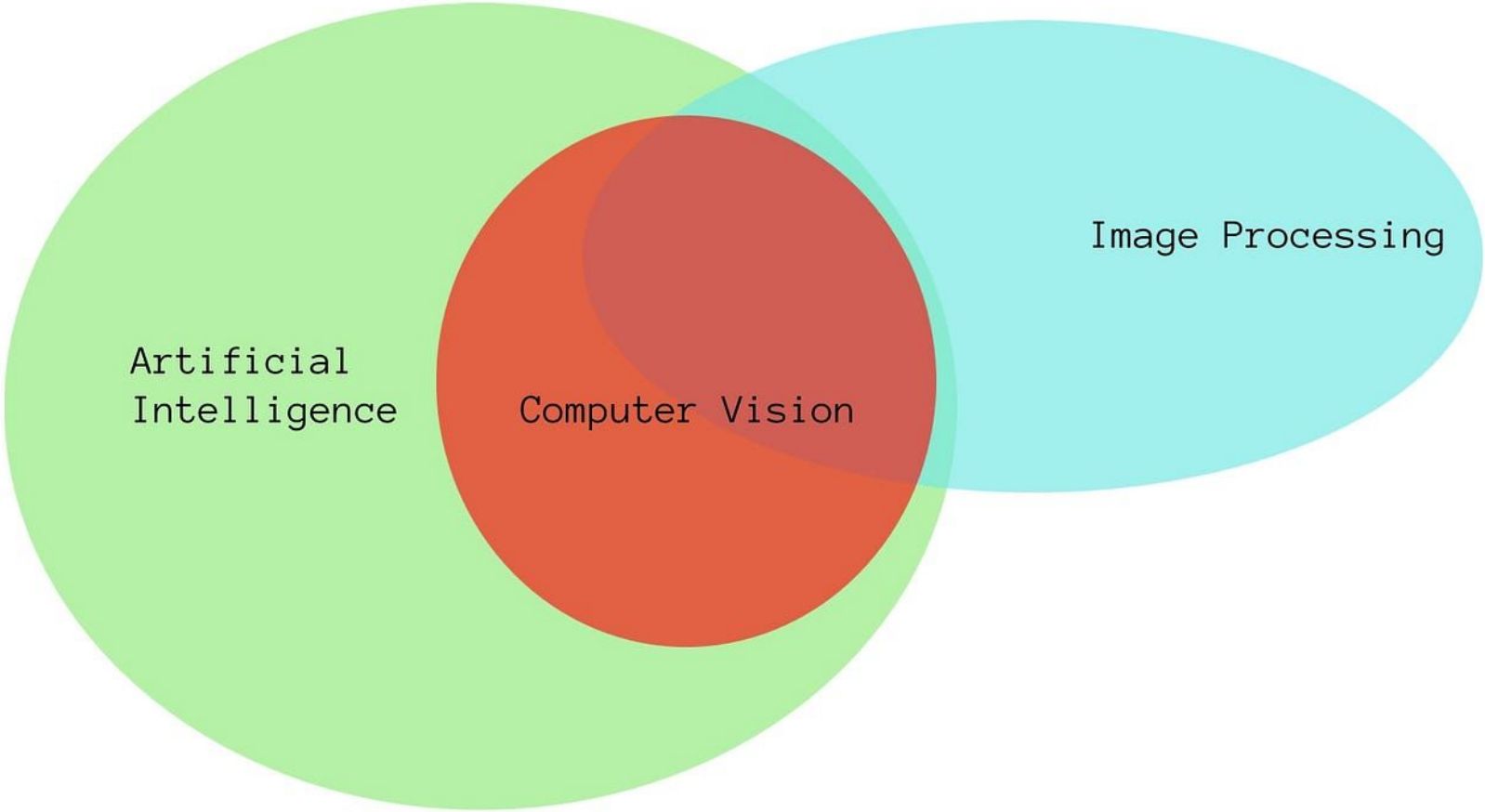
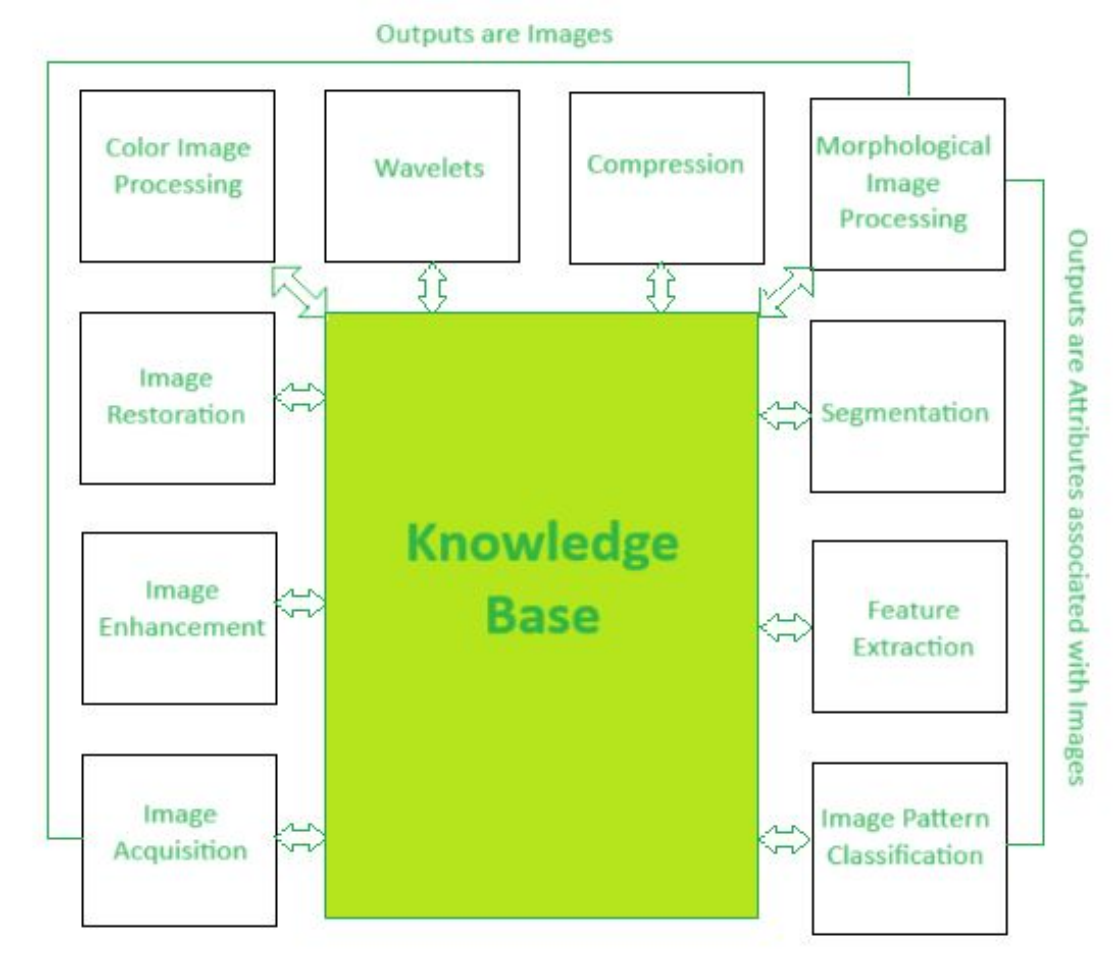


Image Processing
Artificial Intelligence
Computer Vision

Image processing (Revision)

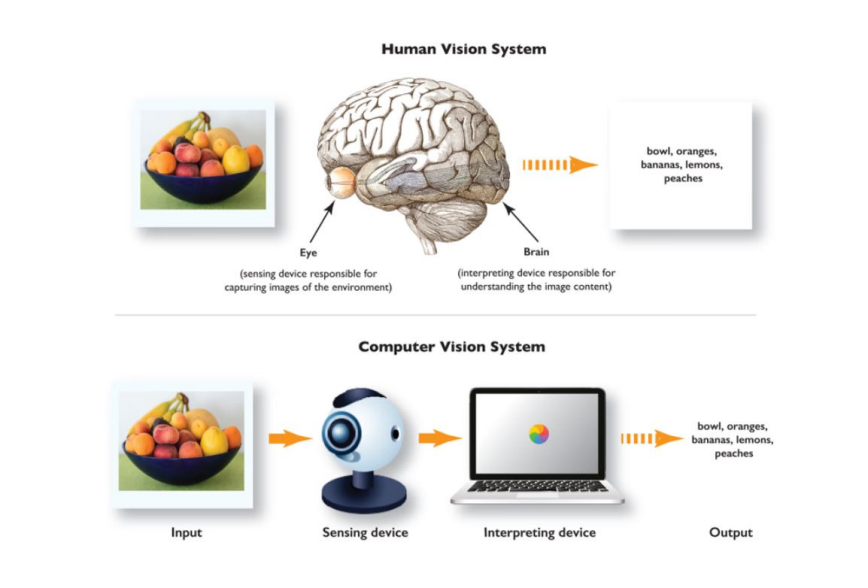
01	Introduction to Image processing	• What is Image Processing? • Image Representation , Its types, size, • Classification of DIP , Fundamental step in DIP
02	Image Enhancement in Spatial Domain	• Basic relation between pixels,Histogram, • Point Processing Techniques, Spatial filters • Resolutions (Spatial & Intensity)
03	Image Enhancement in Frequency Domain	• Frequency Domain enhancement & its properties • Frequency domain filtering (LP, HP, BP)
04	Image Restoration	• What is restoration model? • Degradation types, sources • Notch filter , Inverse filter , winner filter
05	Image Segmentation & Feature Extraction	• Types & techniques of segmentation • Feature Extraction
06	Case Studies and application	• Forest fire Detection • Brain Tumor Detection


Outputs are Images
Color Image Processing
Wavelets
Compression
Morphological Image Processing
Image Restoration
Knowledge Base
Segmentation
Image Enhancement
Feature Extraction
Image Acquisition
Image Pattern Classification
Outputs are Attributes associated with Images

Unit I
Introduction to Computer Vision

What is computer Vision ?

Computer Vision (CV) is a field of **Artificial Intelligence (AI)** that enables machines to **acquire, process, analyze, and understand visual information** from images or videos, **similar to human vision.**



Key Task in Computer Vision

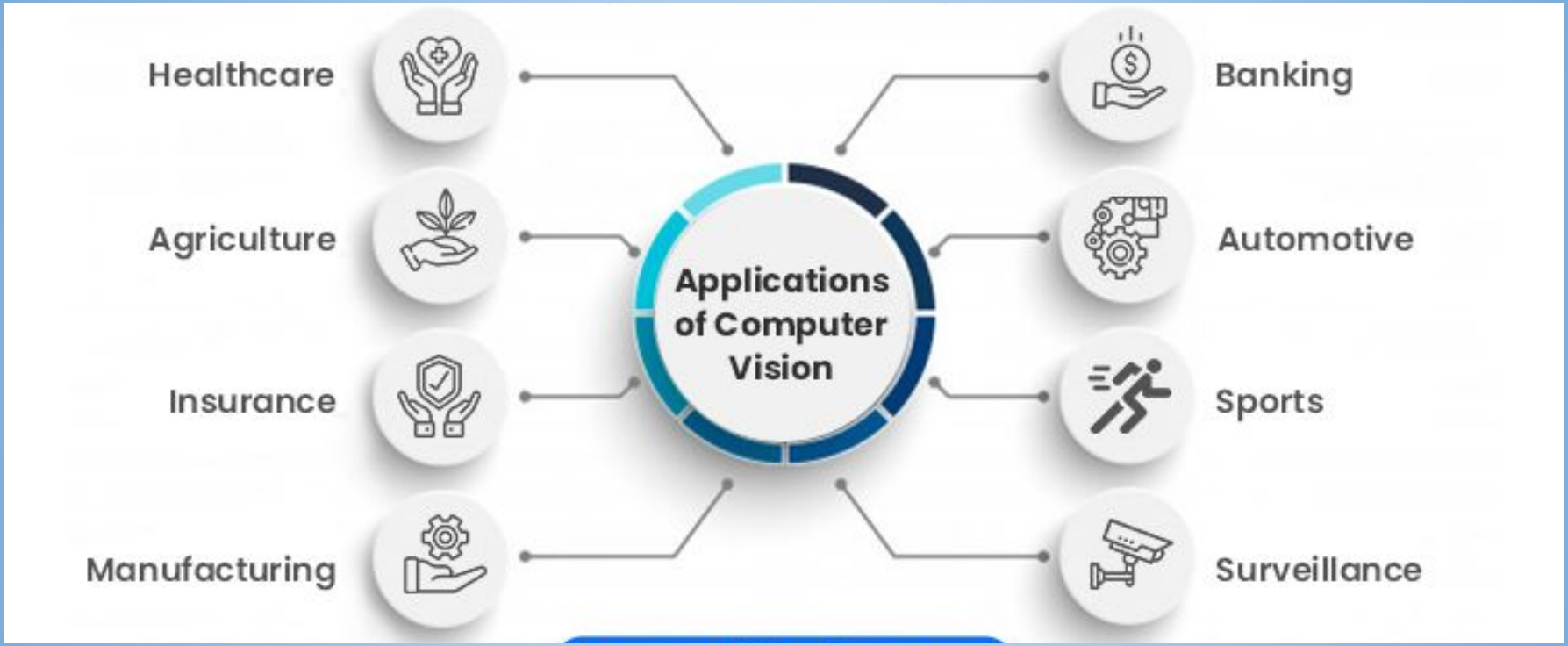
01	Image & video acquisition	• Image & Video sensing • Sampling and quantization
02	Data Pre-processing	• Image Enhancement , filtering, resize etc • Segmentation , thresholding
03	Feature Extraction	• Edge detection, Textures, shapes etc • Motion Analysis
04	Classification , Object Detection & Recognition	• Assigning Label, category, classes • ML, DL algorithms
05	Decision Making	• Action based on visual understanding

Computer Vision vs Image Processing

Image Processing	Computer Vision
Focuses on image enhancement	Focuses on image understanding
Input and output are images	Output is information or decisions
Low-level operations	High-level operation

Application of Computer Vision

- **Healthcare:** Used for disease detection and medical image analysis (X-rays, MRIs).
- **Automotive:** Helps self-driving cars to detect objects, lane keeping and traffic sign recognition.
- **Retail:** It helps with inventory management, theft prevention and customer behavior analysis.
- **Agriculture:** It is used for crop monitoring and disease detection.
- **Security and Surveillance:** It recognizes faces and find suspicious activities in security footage.


Applications of Computer Vision
Healthcare
Agriculture
Insurance
Manufacturing
Banking
Automotive
Sports
Surveillance

Quality Assurance

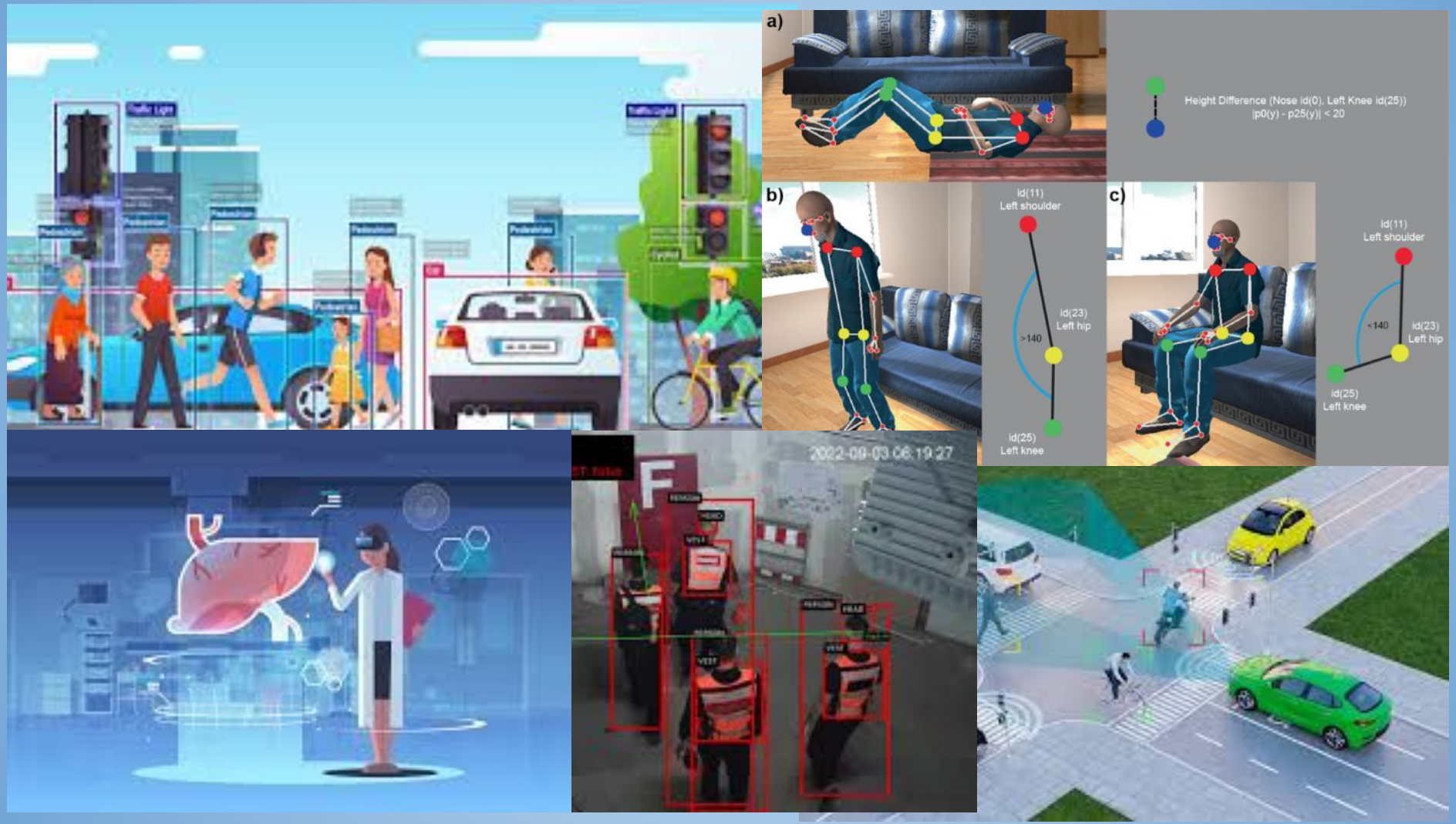
Industrial monitoring

Augmented Realit

Self-driving cars

Robotics

Crop monitoring


a)
Height Difference (Nose id(0), Left Knee id(25))
|p0(y) - p25(y)| < 20
b)
id(11)
Left shoulder
id(23)
Left hip
>140
id(25)
Left knee
c)
id(11)
Left shoulder
<140
id(23)
Left hip
id(25)
Left knee
2022-09-03 06:19:27

Need & scope of Computer Vision

- **Image Classification and Recognition:** Identifying objects, people, actions, or text within images.
- **Object Detection and Tracking:** Locating specific objects in real-time within video streams, essential for robotics and surveillance.
- **Semantic Segmentation:** Classifying individual pixels in an image to understand scene context.
- **3D Reconstruction/Mapping:** Deriving three-dimensional characteristics from 2D images, used in mapping and AR/VR.
- **Image Enhancement/Restoration:** Improving image quality or converting data types (e.g., medical imaging preprocessing).

Need & scope of Computer Vision

- **Revolutionizing Industries:** Powers advancements in self-driving cars, medical diagnostics, agriculture and manufacturing by automating visual tasks.
- **Solving Real-World Problems:** Enhances safety, improves medical imaging and optimizes industrial processes.
- **Improving Accessibility:** It helps people with disabilities through image recognition and sign language translation.
- **Enhancing Consumer Experiences:** It personalizes shopping and improves customer service in retail and entertainment.

Fundamentals of Image Formation

Image formation is the physical and computational process by which a **real-world scene** is converted into a **digital image** that can be processed by a computer.

The mapping of a 3D world object into a 2D digital image plane is called **imaging**.

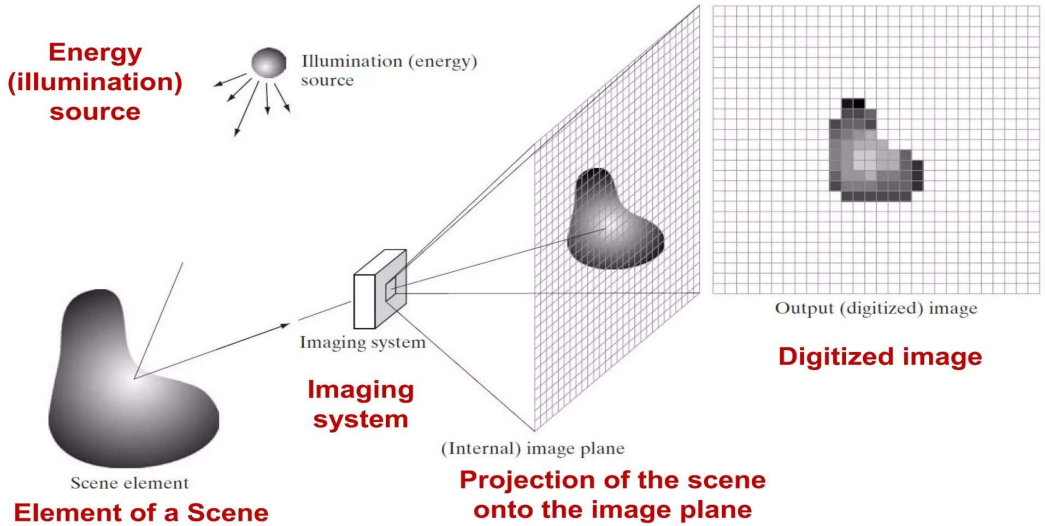
Image formation is **an analog to digital conversion** of an image with the help of 2D **Sampling and Quantization** techniques that is done by the **capturing devices** like cameras.

In computer vision, understanding image formation is essential because image quality directly affects all later tasks such as image quality, feature extraction, detection, and recognition.

Basic Image Formation (Overview)

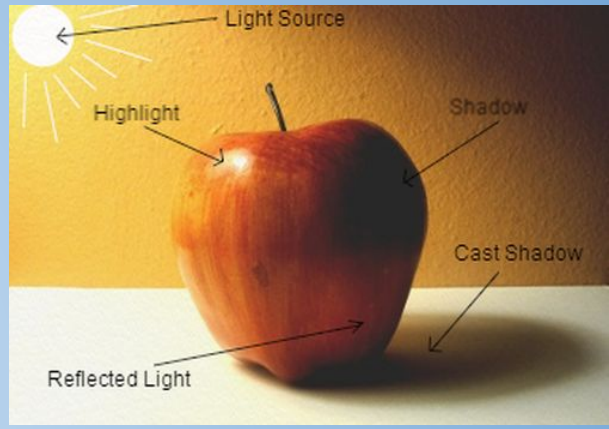
1. Light source illuminates an object
2. Light reflects from the object
3. Camera lens focuses light onto the sensor
4. Sensor converts light into electrical signals
5. Signals are converted into digital images

➢ **Simple Image Formation Model:**



1. Light Source Illuminates an Object

- Light is required to make objects visible. Without light, no image can be formed
- Light may come from (**Types of illumination**) - :
    - **Ambient light** - Natural sources of light present in the environment (sunlight, fire)
    - **Directed light** - Artificial sources of light (bulbs, LEDs, lasers, flashes)

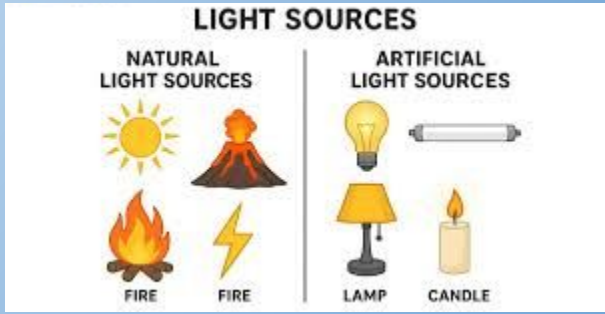


Illumination L(x,y) - Represents the amount of light incidents on the objects. It changes based on the lighting source, such as time of day, shadows or different lamps.

Effect of Illumination

- Intensity of light affects **brightness**
- Direction affects **shadows and texture**
- Color of light affects **image color**

Poor illumination results in **dark images** and **loss of details**.



2. Light reflects from the Object

When light strikes an object, it interacts with the surface and gets:

- Reflected
- Absorbed
- Scattered

Computer vision systems mainly rely on **reflected light**.

Reflectance R(x,y) - Represents the intrinsic properties of objects in the scene (color, texture,material,smoothness) . this remains the same regardless of the lighting condition.

A white object reflects more light → appears brighter

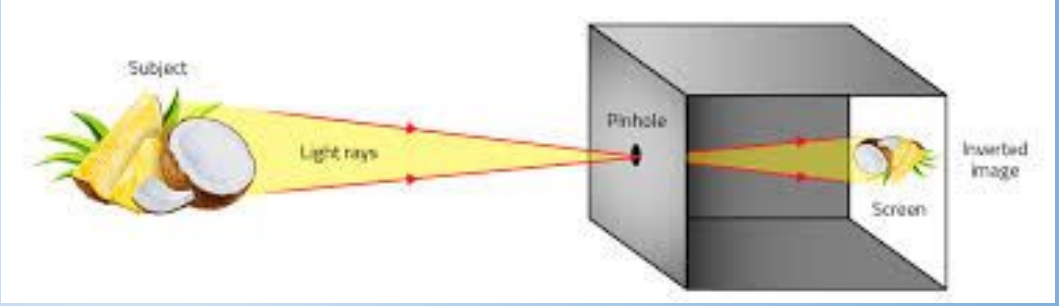
A black object absorbs more light → appears darker

3. Camera lens focuses light onto the Sensor

The camera lens collects reflected light and focuses it onto the sensor.

The lens controls:

- Focus
- Sharpness
- Amount of light



Pinhole Camera Model
(Simplified Lens Model)

Lens Components

- Aperture (controls light amount)
- Focal length (controls magnification)
- Optical elements (focus rays)
- Focal length (f): Distance between lens and sensor
- Aperture: Opening size of lens

Effect of Lens

- Incorrect focus → blurred image
- Lens distortion → curved lines

Example - Autofocus camera adjusts lens to keep objects sharp.

4. Sensor Converts light into electrical signals

The image sensor is a **light-sensitive electronic device** that converts light into electrical signals.

Types of Sensors

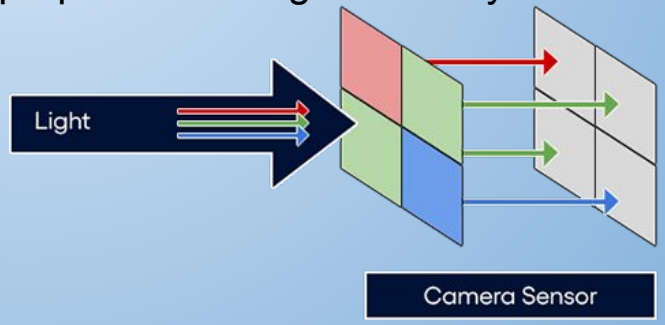
- CCD (Charge Coupled Device)
- CMOS (Complementary Metal Oxide Semiconductor)

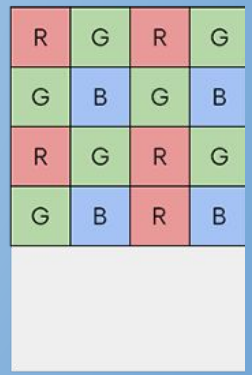
Working Principle

- Sensor is made of millions of tiny light-sensitive cells (pixels)
- Each pixel generates an electrical charge proportional to light intensity

Example

- More light → stronger electrical signal
- Less light → weaker signal

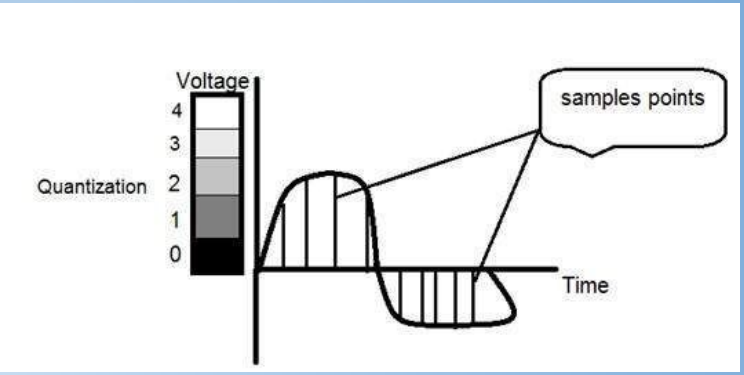




5. Signals are converted into digital Images

The electrical signals from the sensor are:

1. Amplified
2. Sampled
3. Quantized
4. Stored as digital values



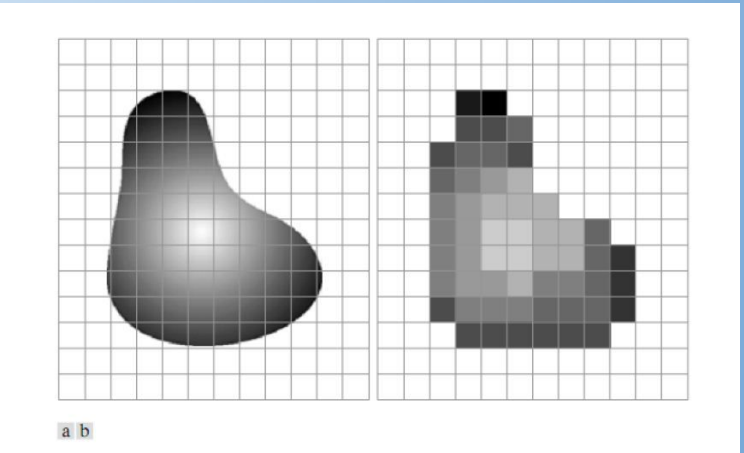
Pixel Value Representation

- Grayscale image: values from 0–255
- Color image: RGB values for each pixel

If pixel intensity = 0 → Black

If pixel intensity = 255 → White

If pixel intensity = 128 → Gray



Numerical

Mathematical Model of Image Formation

Fundamental principle in image formation an observed image f(x,y) is modeled as the product of two components illumination L(x,y) and reflectance R(x,y)

Image intensity at (x, y) is given by:

$$\mathbf{I(x, y) = L(x, y) \times R(x, y)}$$

Where:

- L(x, y) = Illumination
- R(x, y) = Reflectance ($0 \leq R \leq 1$)
- I(x,y) = Image intercity at (x,y)

Example : Illumination = 200, Reflectance = 0.6 what will be the pixel value ?

Qu 1. If Illumination L = 150, Reflectance R = 0.8. Find: Image intensity I ?

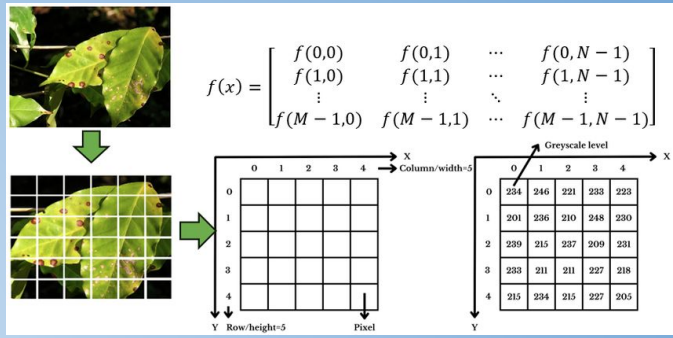
Qu 2. Pixel intensity I = 90, Illumination L = 180. Find: Reflectance R ?

Qu 3. If Illumination varies from 100 to 200, Reflectance = 0.6. Find: Minimum and maximum pixel intensity ?

Qu 4. Why does the same object appear different under different lighting?

Answer: Because illumination L(x, y) changes while reflectance R(x, y) remains constant.

Digital Image Representation



An **image** is defined as a **two-dimensional function.**

Digital Image = F(X,Y)

Where, **X and Y are spatial coordinates**, and the **F amplitude at coordinates (x, y)** is called the **intensity** or gray levels of the image at that point.

A digital image is composed of **M** rows and **N** columns (Matrix). Each element of this matrix is called **image element**, **picture element, pixel or pel.**

No. of pixels in image = width (M) x height (N) pixel

Example : If image is of 1024 x 1024 image then it contain 1048576 pixels

Intensity levels or gray levels of image depends on **bits** of image.

No. of gray level (L) = 2^k

where, k - no. of bits (1,2,3,....)

Bits	No. of gray level	Intensities
1	2	[0 , 1]
2	4	[0 , 1 , 2 , 3]
3	8	[0 , 1 , 2 , 3 , 4 , 5 , 6 , 7]
8	256	[0 , 1 , 2 .... 255]

Color Space

Color spaces are mathematical models used to **represent and organize the color channels** in digital images.

Color space define **how colors are formed from primary colors and stored**,allowing computers to process and manipulate color information efficiently.

Each color space has a specific purpose and the choice of color space depends on the task.

An image is simply a 3D matrix:

Image = Height X Width X Channels

Example:

- RGB image → 3 channels
- Grayscale → 1 channel

Some common color spaces are as follow -

1. **RGB (Red, Green, Blue):**

   RGB color model represents colors using three **primary colors: red, green and blue**.

   Each pixel = combination of R,G,B $\in$ [0,255]

   It is most common color space used for display images on screen & working with digital images.

   Note: While working with OpenCV -

   OpenCV uses **BGR** instead of **RGB** color space by default.

Example: pixel = (120,200,150)

This means R = 120 , Green =200, Blue = 150

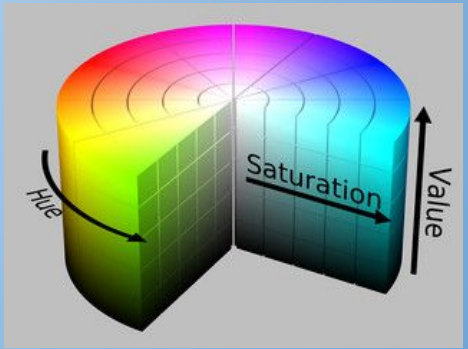
- If pixel = (255,0,0) then it will be red colored pixel.
- If pixel = (255, 255, 255) then it will appear white colored pixel.
- If pixel = (0, 0, 0) then it will appear black colored pixel

2. HSV/ HSL (Hue, Saturation, Value/Lightness)

It's based on the human perception of color and it splits color information into three components:

- **Hue (0 to 179)** : Represents the color itself (e.g red, blue, green).
- **Saturation (0 to 255)** : Shows the vibrancy of the color.
- **Value (0 to 255)** : Defines the brightness of the color.

HSV separates color information (hue) from brightness. Hence, it is useful for color based segmentation/ detection, object tracking.



3. Grayscale

It is **single intensity channel** contains only shades of gray with each pixel representing a different intensity level from black to white.

It is used when color information is not needed such as in edge, face detection, medical imaging, image thresholding tasks or faster processing.

Mathematical formula ;

$$\textbf{Gray} = 0.299R + 0.587G + 0.114B$$

Note - These weights are based on human eye sensitivity.

Example - Find gray value of color pixel with R=100, G=150, B=200 ?

- **Why Different Color Spaces Matter?**

  Choosing the right color space is critical because it can significantly improve the robustness of algorithms.

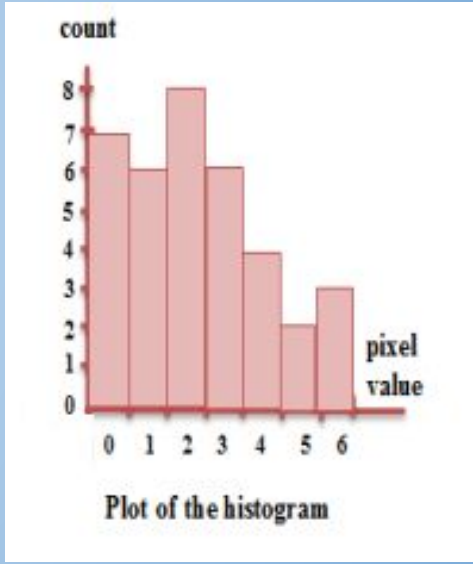
  For example, RGB is sensitive to lighting changes, making HSV more effective for tracking objects under varying illumination.

Histogram

What is an Image Histogram?

An image **histogram** is a **graphical representation of the distribution of pixel intensity values** in an image.

$$h(r_k) = n_k$$



Where, r_k - Gray level of k

n_k - frequency of gray level k

It shows:

- X-axis - Intensity values (0–255 for 8-bit image)
- Y-axis - Frequency (number of pixels)

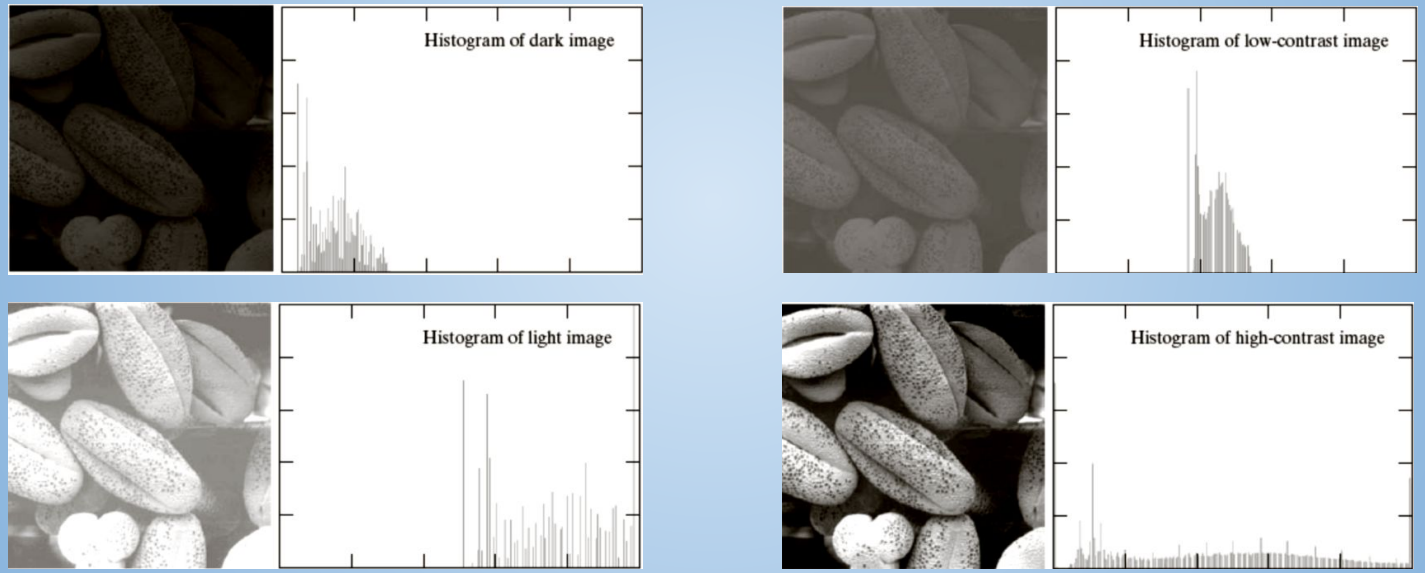
Histograms **reveals** a lot about image like - **Over and under exposure of light, Brightness , Contrast , Dynamic Range (intensity levels) etc**

Qu. Draw histogram of the image.

$$\begin{bmatrix} 10 & 20 & 30 & 40 & 50 \\ 20 & 30 & 40 & 50 & 60 \\ 30 & 40 & 50 & 60 & 70 \\ 40 & 50 & 60 & 70 & 80 \\ 50 & 60 & 70 & 80 & 90 \end{bmatrix}$$

➢ Types of Image based on histogram

By using histogram we able to see immediately: Whether an image is dark, bright, high contrast or low contrast image



Point processing operations can be used **to alter histogram** for **image enhancement**.

➢ Contrast Stretching :

$$contrast = I_{\max} - I_{min}$$

Contrast Stretching **improves image contrast** by linearly **expanding the range of pixel intensity values to cover the entire possible range.**

Contrast stretching involves applying a **linear transformation** to each pixel intensity value.

$$S = \left(\frac{(r - r_{\min}) \times (s_{max} - s_{min})}{(r_{\max} - r_{min})} \right) + s_{min}$$

Where:

- r = input pixel intensity
- s = output pixel intensity
- r_{min},r_{max} = minimum & maximum pixel values in input image
- s_{min},s_{max} = desired output range (typically 0 and 255 for 8-bit images)

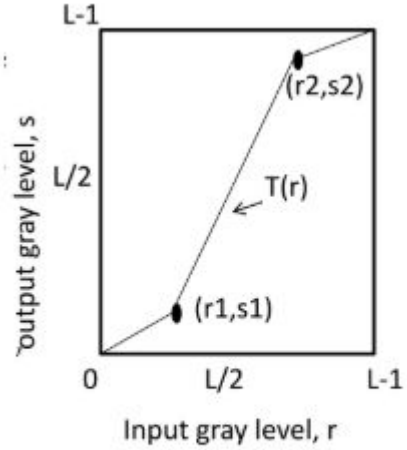
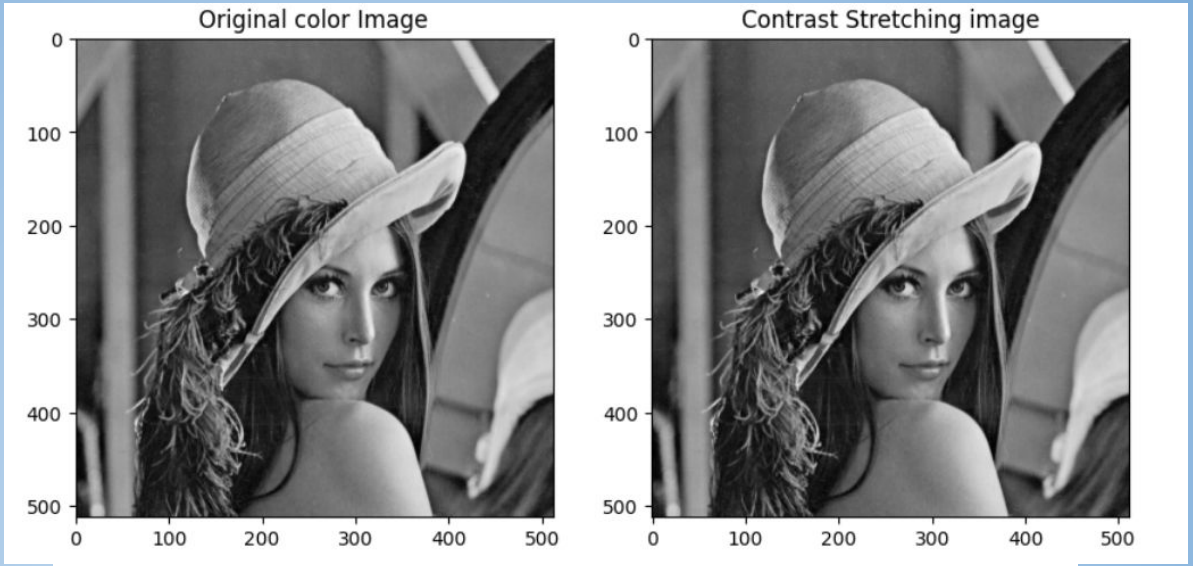
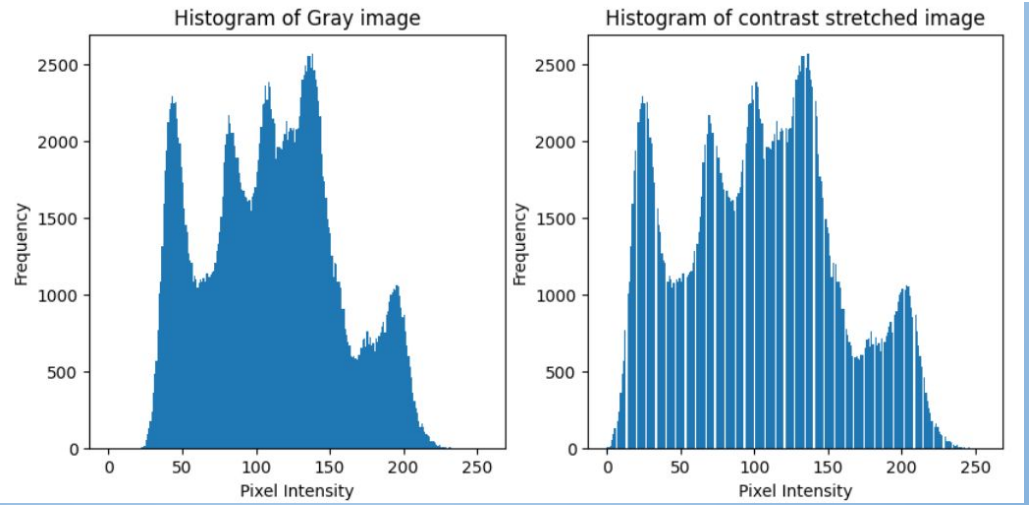


Fig: transformation used for contrast stretching



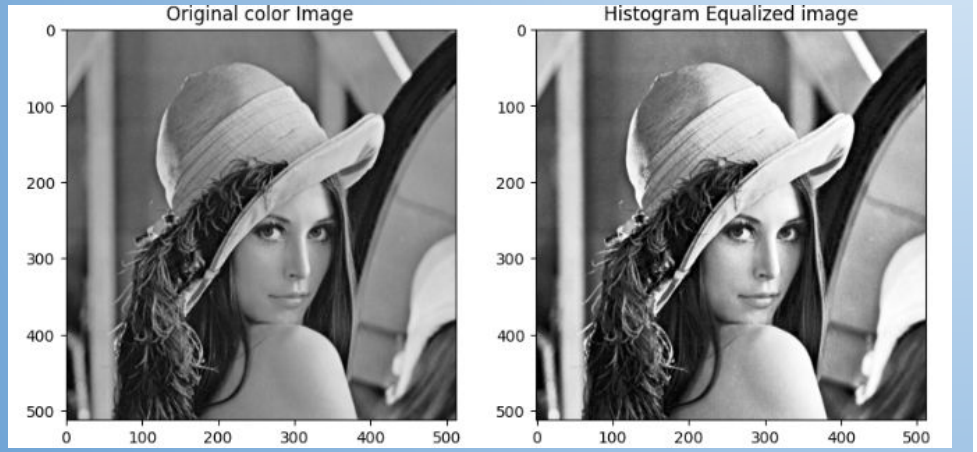


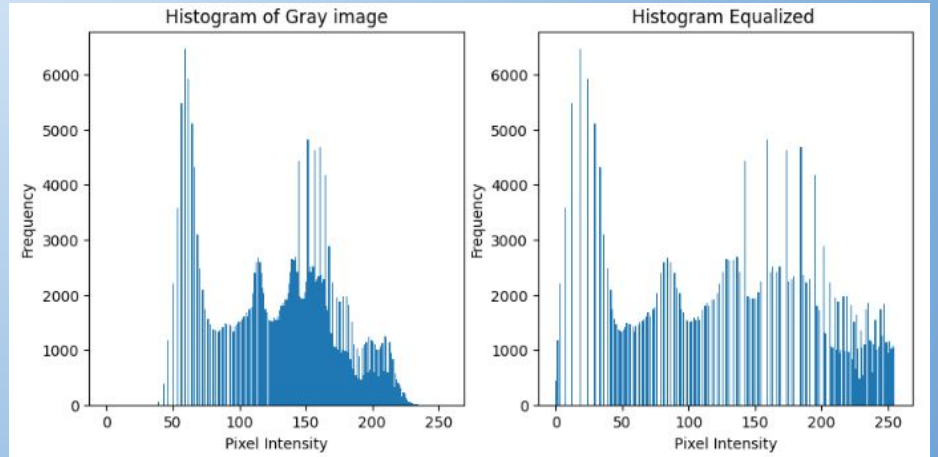
➢ Histogram Equalization

This method is **use to improve an image contrast** by redistributing its pixel intensity values to create a more uniform distribution.

It uses the **Cumulative Distribution Function (CDF)** of the histogram as a mapping function.

$$S = (L-1)\sum_{i=0}^{k} p(i)$$





Please refer **Introduction to Image Processing ppt (Sem 3)**

Topics to refer -

- **Sampling & Quantization, Image representation (**Unit 1 : Introduction to Image Processing **(Page no. 18 - 25, 35 - 41, 47 - 55)**
- **Histogram & Related operation (**Unit 2 : Image Enhancement in Spatial Domain **(page no. 32 - 55)**

Computer Vision Lab Practice

➢ **Required Environment**

Computer Vision project environment requires

- python 3+
- Numpy , OpenCV Library etc
- Other - matplotlib

OpenCV (Open Source Computer Vision Library) is an open-source library for image processing and computer vision tasks like

- Image and video processing
- Feature detection
- Object detection
- Machine learning support

- **Install Required Libraries**

To install libraries > Go to Command prompt

pip install numpy

pip install opencv-python

pip install matplotlib

➢ Import required Libraries

To install libraries > Go to CMD

```
import numpy as np
import cv2
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
```

➢ Load image using - imread( )

This method loads an image from the specified file and function return a NumPy array if the image is loaded successfully.

Using OpenCv -

```
# Opencv read image in BGR color space
img= cv2.imread("Path/FileName")
```

Using matplotlib -

```
# matplotlib read image in RGB color space
img= mpimg.imread("Path/FileName")
```

Sample image

➢ Display Image - imshow( )

Using OpenCv - It creates a window with a specified title and displays the given image matrix.

```
cv2.imshow("Title", image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Using matplotlib - It used to display images in a plot.

```
plt.imshow(image)
plt.show()
```

- **Store image - imwrite( )**

  imwrite( ) function used to save an image to storage in any supported format such as JPG, PNG, or BMP. It saves the provided image to the given filename, and the format is determined automatically by the file extension.

```
cv2.imwrite("saved_image.jpg", img)
```

- **To get dimensions of image -**

  image.shape is an attribute of the image array that returns a tuple representing the dimensions of the image

```
img1= cv2.imread("Filename", 1)     # 1-Read image in color
img2= cv2.imread("Filename.png",0)     # 0-Read imag in gray
print(img1.shape)     >>> (217, 164, 3)
print(img2.shape)     >>> (217, 164)
```

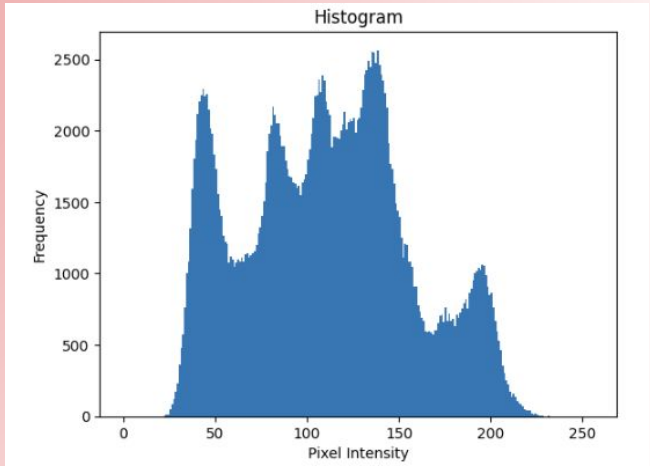
➢ Plotting Histogram of image

- Histogram using OpenCV - **cv2.calcHist( )**

```
hist = cv2.calcHist([img],[0],None,[256],[0,256])
```

- Histogram using NumPy- **np.histogram( )**

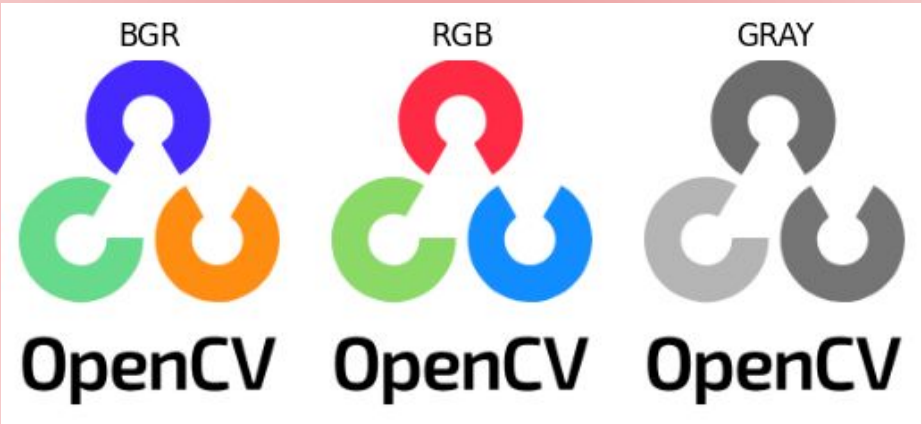
```
plt.hist(img3.ravel(),bins=256,range=[0,256])
plt.show()
```



➢ Convert color space of image

```
img_rgb = cv2.cvtColor(image,cv2.COLOR_BGR2RGB)
```

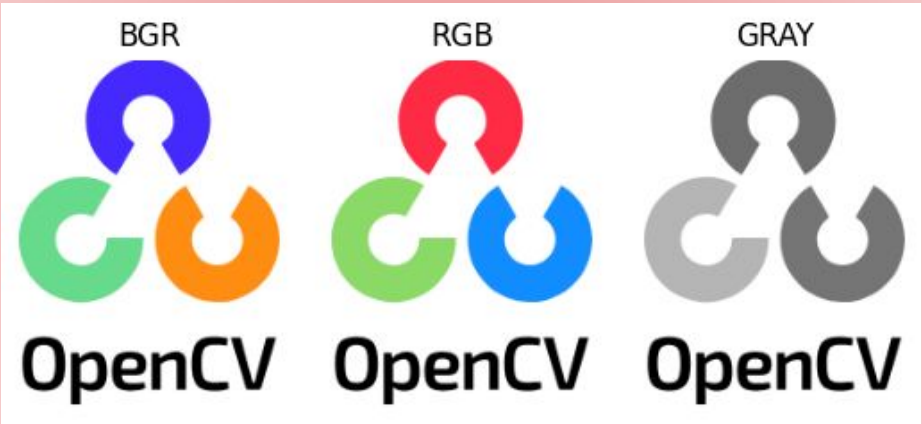
```
img_gray = cv2.cvtColor(image,cv2.COLOR_BGR2GRAY)
```



➢ Convert color space of image

```
img_rgb = cv2.cvtColor(image,cv2.COLOR_BGR2RGB)
```

```
img_gray = cv2.cvtColor(image,cv2.COLOR_BGR2GRAY)
```



- **Thresholding**

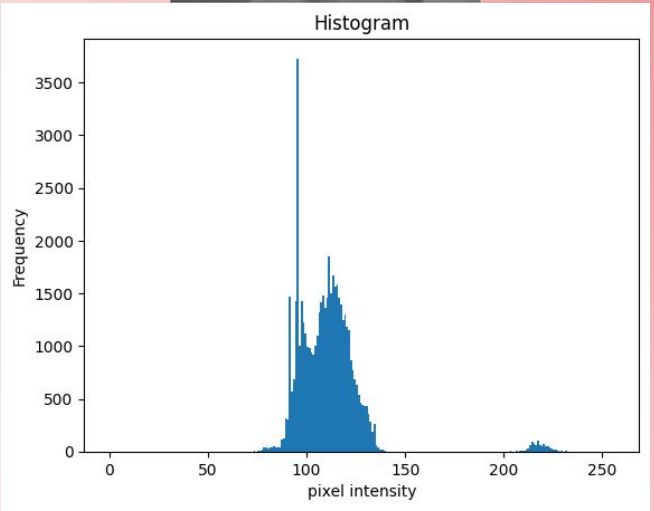
```
ret, thresh = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
```

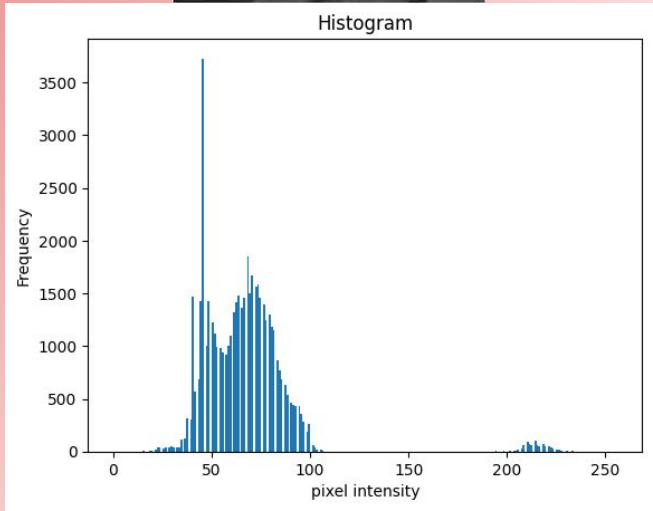
- **Negative image**

```
negative = cv2.bitwise_not(thresh)
```

➢ Contrast Stretching

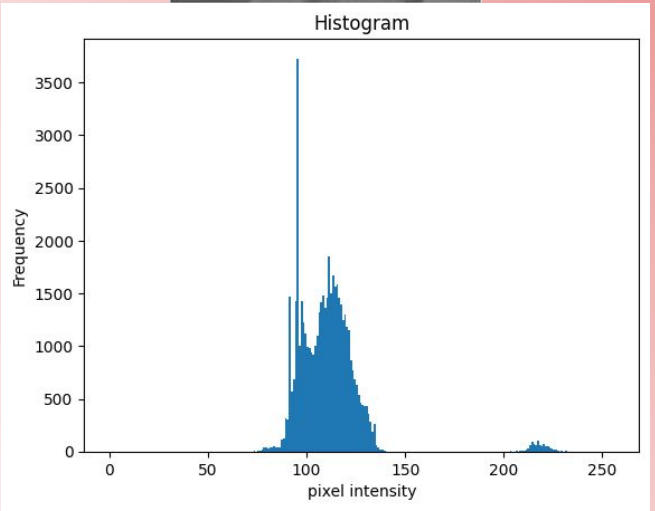
```
stretched = cv2.normalize(gray, None, 0, 255, cv2.NORM_MINMAX)
```

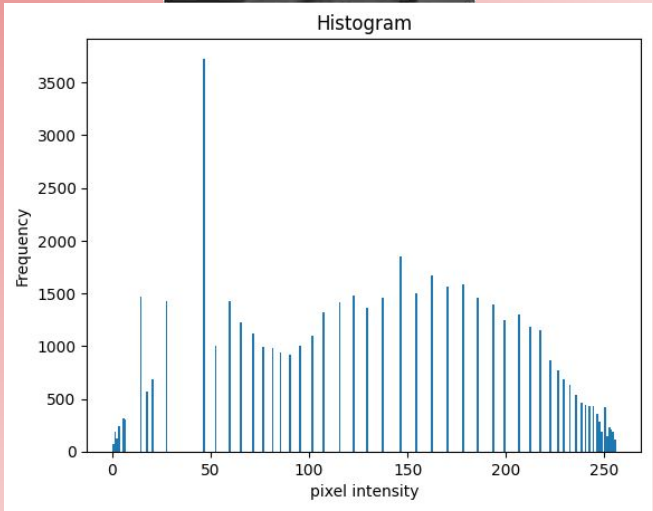




➢ Histogram Equalization

```
equalized = cv2.equalizeHist(img_gray)
```





Assignment: (Programming) Deadline: 28th Feb

Write python functions to -

1. Plot histogram of image
2. Color transformation RGB to GRAY
3. Thresholding
4. Image Negation
5. Contrast Stretching
6. Histogram Equalization

Note: Use subplot to display input image, output image (using builtin function) & output image (without using inbuilt function) also plot histogram for histogram based methods

Assignment: Deadline - 2nd March

1. What is Computer Vision? Explain different pipelines in Computer Vision.
2. How is an image formed? Explain with a neat labeled diagram.
3. If Illumination = 200 and Reflectance = 0.6, what will be the pixel value?
4. If Illumination varies from 100 to 200 and Reflectance = 0.6, find:
    a. Minimum pixel intensity
    b. Maximum pixel intensity
5. Write a note on different color spaces and their uses.
6. Find the gray value of a color pixel with R = 255, G = 10, B = 100.
7. Write a note on histogram. Also explain how histogram help in image analysis
8. Explain different point processing methods based on histogram to enhance an image.
9. Why is OpenCV used?
10. Explain the use of the following functions:
    a. imread()
    b. imshow()
    c. hist()
    d. cvtColor()
11. Write pseudocode to plot the histogram of an image.
12. Write algorithm for contrast stretching